

Operations Nerd — Updated HLD (v2)

A senior's read of the v1 proposal. Trimmed, opinionated, ready to share.

Senior researcher · 2026-07-25 · for the junior author (primary) and Prof. Sasha

v1 in one breath

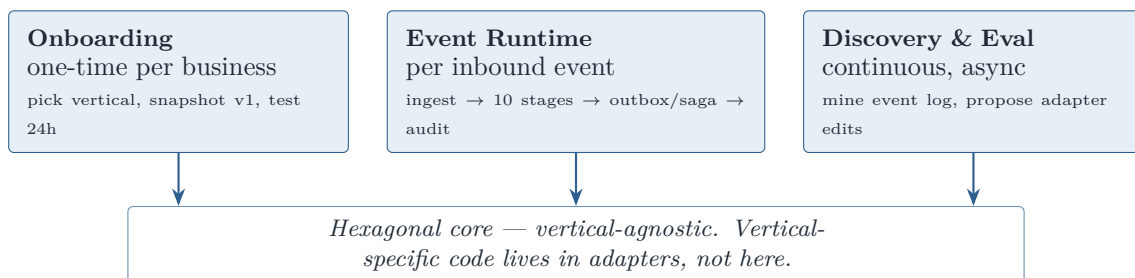
Right thesis (config-first, shared ontology, vertical adapter). Thin substrate: four-primitive ontology, no data model, no multi-tenant story, no reliability layer for side effects, no HITL design, no eval plan, and the nine-stage pipeline conflates one-time setup with per-event processing.

v2 in one breath

Same thesis, real engineering. Adds a two-layer ontology, Postgres RLS multi-tenancy, transactional outbox + saga, durable execution on Temporal+LangGraph, an explicit HITL gate with three-signal confidence, a git-shaped snapshot model, and a continuous discovery loop. The nine stages split into a *one-time onboarding pipeline* and a *per-event runtime pipeline*.

1 Three planes, three cadences

The runtime is not one pipeline, it's three. v1's stage 7 was actually onboarding, not a per-event stage; v2 makes that explicit.



The core follows *Ports & Adapters* (Cockburn): vertical-specific code is an adapter. Anything that differs per business lives in the saved-state snapshot; anything that doesn't stays in the core.

2 Per-event runtime pipeline (10 stages)

v1's stages 7–9 were actually onboarding, not runtime. v2 splits them out cleanly. The runtime is ten stages end-to-end, every stage writes an append-only **Event** row, and the last two stages run in parallel after the action lands.

#	Stage	What it does (and the failure guard)
1	Ingest	Webhook / polling / CSV arrives. Idempotency row by (tenant , source , source_event_id). Dedupe.
2	Validity Gate	Confirm legitimate event (not spam, not malformed). LLM-as-judge + rule prefilter.
3	Ontology Conversion	Map to upper-core entities (Party / Engagement / Agreement / Transaction / Obligation / Event). JSON-Schema validated.
4	Context Extraction	Read prior records, related entities, calendar, payment status. Read-only Temporal activity.
5	Function Classification	Route to relationship / sales / outreach / accounting. Constrained LLM + rule fallback.
6	Compliance & Safety	Pluggable: OPA/Rego rules, LLM-as-judge, HITL escalation. All decisions logged to audit.
7	Action Drafting	Produce typed action object (SendEmail , PostInvoice , ...). Pure object, never executed.
8	HITL Gate	<i>auto</i> / <i>requires_approval</i> / <i>block</i> per action type. Three-signal confidence threshold.
9	Action Execution	Outbox row + state change in <i>one</i> DB transaction. Idempotency four-tuple. Saga compensations.
10a	Audit Log	Append Event row (XES-extended). Drives discovery + compliance audit + drift monitor.
10b	Live Feedback	SSE stream to the Web Console, per event_id , stage-by-stage.

3 Saved-state snapshot (the actual deliverable)

Per-business adapter state, versioned like git. Immutable blobs in object storage, pointer + metadata in the DB, hash-checked on every read. *Anything that differs per business lives here; anything that doesn't lives in the core.*

```

snapshot = {
  schema_version "opsnerd.snapshot.v2",
  snapshot_id "snap_<uuid>",
  parent_snapshot_id "snap_<uuid>" | null, // fork / rollback
  tenant_id "<uuid>",
  provenance "onboarding" | "discovery_proposal" | "manual_edit" | "rollback",
  ontology { ... lower-core per vertical ... },
  policies { ... OPA bundle, action-type approval defaults ... },
  action_templates { ... typed action shapes per use case ... },
  integrations { ... scopes only; tokens live in Vault ... },
  hash sha256(canonical_json), // integrity
}

```

Rollback = load the parent snapshot; in-flight events resume from their Temporal checkpoint with the parent state loaded. History is append-only.

4 The three things that make it production-realistic

Without these, the junior's pipeline is a research demo. With them, it's shippable. The literature is unambiguous on each.

1. Transactional outbox + saga for every side effect. CRMArena-Pro (Salesforce, 2025) puts the best LLM agents at 58% single-turn / 35% multi-turn on real CRM tasks. Production-agent studies (Fiddler, gravity.fast, 2026) put real-world failure rates at 70–95%, with 88% of demo-passing agents failing in production. The fix isn't a better prompt, it's a separate dispatch layer that

guarantees the state change and the external action are atomic. We write the outbox row *in the same transaction* as the state change; a separate drainer dispatches. Four-tuple idempotency key (`agent_run_id`, `step_id`, `tool_name`, `business_scope`); both forward and compensation actions are idempotent under the same key.

2. HITL as a first-class subsystem, not a footnote. v1 mentioned approval in §4 prose. v2 makes it stages 7–8 with a three-signal confidence threshold (LLM + rule coverage + historical accuracy on this action type), default policy *auto/requires_approval/block* on a reversibility × risk matrix, and a *promote-to-policy* path that turns a one-off decision into a standing rule. Practitioner data (dev.to, 2026) shows HITL reduces critical error rate from 23% to 5.1% on the same agent workflow. The human override rate is the most reliable leading indicator of system quality, alert on trend.

3. Durable execution on Temporal + LangGraph. Per LangChain’s 1.0 release (Oct 2025) and the 2026 production consensus: *Temporal* for the outer workflow (retries, compensations, multi-day waits, cross-service), *LangGraph* for the inner agent reasoning (cyclical control flow, conditional routing, checkpoint per node to Postgres). The outbox is the bridge between "the agent decided" and "the external system was called."

5 Data model and ontology in one breath

Two-layer ontology. Eight upper-core entities in the core (Party, Party Relationship, Engagement, Agreement, Transaction, Transaction Line, Obligation, Event), per-vertical lower-core sub-classes in the snapshot. Every table has `tenant_id` + Postgres RLS (`FORCE ROW LEVEL SECURITY`, `app.current_tenant` GUC set on the connection). Outbox, saga, approvals, `audit_log`, and snapshots live in the same DB. Integration tokens are encrypted, never in snapshots.

The two-layer pattern is taken straight from O-CREAM-v2 (Cairns et al., 2010) and CURIE-O (Sales et al., 2021), the academic CRM ontologies for SMEs. v1’s four primitives can’t model real CRM relationships, audit trails, or event sourcing. Eight upper-core entities can.

6 Open questions for the junior

v3 is gated on answers to these. Pick one, defend the choice in two paragraphs.

- 1. First vertical.** My pick is real estate: lower regulatory bar than clinic, higher willingness-to-pay than generic sales, and the v1 walkthrough already uses it. Clinic is second-best if you want to lead with the higher revenue ceiling.
- 2. Build vs buy the runtime substrate.** My pick is Temporal Cloud + LangGraph 1.0 (managed) to remove a class of operational burden in v1. Temporal OSS + custom checkpointing is cheaper at scale but eats engineering time. Decide before you write a line of runtime code.
- 3. First saved-state authoring path.** My pick is discovery-loop-first: mine the business’s first 90 days of events to propose a starter snapshot, then the owner reviews and edits in the Web Console. Pure hand-authored questionnaire is the alternative.
- 4. Approval default for `send_external`.** My pick: *requires_approval* by default for 30 days, then owner-configurable to *auto* per action type after a clean track record. Never default to *auto*.
- 5. Relationship to CustomNerd.** My pick is to publish v2 as a separate artifact with explicit cross-references in the dissertation chapter, not as a CustomNerd extension. The engineering substrate (durable execution, outbox/saga, RLS, snapshot model) is the difference, and conflating them will confuse readers.

Sources: CRMArena-Pro (Salesforce, 2025) · O-CREAM-v2 · CURIE-O · PMAx/pm4aa (2026) · LangGraph 1.0 · Temporal · Replit snapshot engine · Postgres RLS production patterns · Fiddler / gravity.fast / armalo 2026 practitioner studies. Failure-mode catalog, full RLS schema, and eval plan are in the companion `Operations-Nerd-HLD-v2.md`.